

Installation of PCXMC:

- Avoid installing to Program Files or Program Files (x86).
- If you do install to above and find you're having problems, try copying the entire PCXMC installation folder to somewhere else, like your documents.
- Tends to cause errors related to permission/administrative rights. You can fix them, but it's easier to avoid them.

If you have a 64-bit OS you may need to modify the macros:

- Open “AutocalcRotation-sheet.xls”
- Enable develop options and open the VBA editor for the first macro
- Replace:

```
Private Declare Function WaitForSingleObject Lib "kernel32" (ByVal _  
    hHandle As Long, ByVal dwMilliseconds As Long) As Long  
  
Private Declare Function CreateProcessA Lib "kernel32" (ByVal _  
    lpApplicationName As Long, ByVal lpCommandLine As String, ByVal _  
    lpProcessAttributes As Long, ByVal lpThreadAttributes As Long, _  
    ByVal bInheritHandles As Long, ByVal dwCreationFlags As Long, _  
    ByVal lpEnvironment As Long, ByVal lpCurrentDirectory As Long, _  
    lpStartupInfo As STARTUPINFO, lpProcessInformation As _  
    PROCESS_INFORMATION) As Long  
  
Private Declare Function CloseHandle Lib "kernel32" (ByVal _  
    hObject As Long) As Long
```

- With

```
Private Declare PtrSafe Function WaitForSingleObject Lib "kernel32" (ByVal _  
    hHandle As Long, ByVal dwMilliseconds As Long) As Long  
  
Private Declare PtrSafe Function CreateProcessA Lib "kernel32" (ByVal _  
    lpApplicationName As Long, ByVal lpCommandLine As String, ByVal _  
    lpProcessAttributes As Long, ByVal lpThreadAttributes As Long, _  
    ByVal bInheritHandles As Long, ByVal dwCreationFlags As Long, _  
    ByVal lpEnvironment As Long, ByVal lpCurrentDirectory As Long, _  
    lpStartupInfo As STARTUPINFO, lpProcessInformation As _  
    PROCESS_INFORMATION) As Long  
  
Private Declare PtrSafe Function CloseHandle Lib "kernel32" (ByVal _  
    hObject As Long) As Long
```

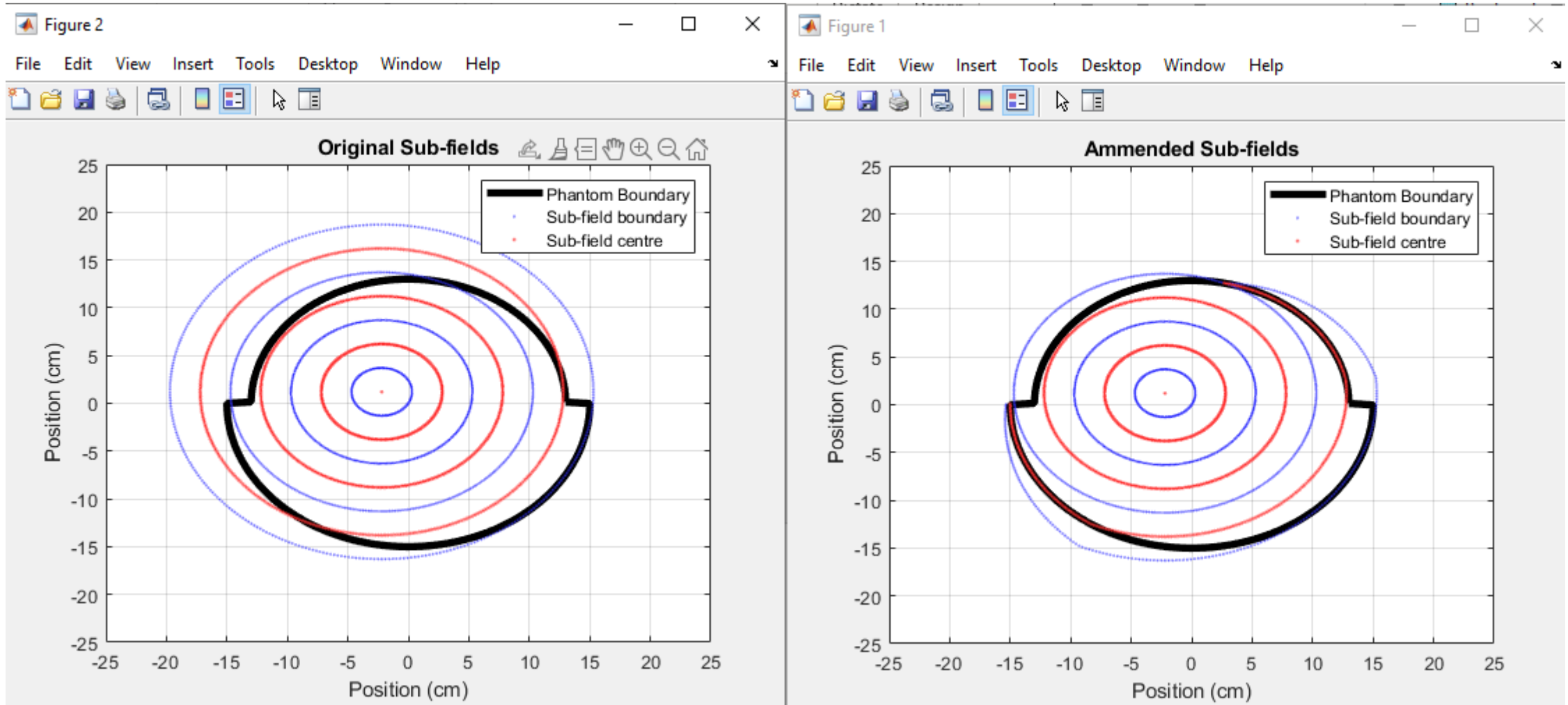
- We’re just adding “PtrSafe” before “Function” to the three blocks located towards the bottom of the first section

MATLAB code has 5 functions and 1 script:

- “Main” is the primary script which is used to input your data and export data for PCXMC.
- The 5 functions include:
 - “findIntersections”: Finds intersections between lines and ellipses
 - “reScale”: Upscales or downscales resolution of input data using linear interpolation
 - “subFieldIntersection”: Finds the boundary between adjacent subfields
 - “subFieldIntersections”: Finds the boundaries between arrays of adjacent subfields
 - “withinEllipse”: Checks if the point is within the ellipse
 - “WrapTo360”: Ensures angles in degrees are within ± 360 . e.g. 540 becomes 180
- Majority of code is commented with descriptions
- Let me know if you have any questions or are getting errors.
- I’ve sent it to you pre-filled with dummy values, I would probably run the main script as is and it should output graphs immediately, identical to those on the following page

MATLAB Output

- If plotting is enabled it will show the input fields and amended fields for PCXMC



MATLAB “Main” Script

```
%%%%%%%%%%%%% INPUT DATA  %%%%%%%%%%%%%%

%Phantom position details
iso = [-2.2; 1.2]; %Isocentre coordinate (x,y), relative to the geometrical centre of the phantom in the transverse slice (cm)
z = 0; %Sup-Inf offset (cm)
arms = 0; %Binary value used to represent if the arms are included in the phantom within PCXMC. 0 = no, 1 = yes.

%Scan parameters
UID = 1; %Dummy value for patient ID
kV = 125; %Tube voltage
FRD = 100; %Distance from the source to the reference point (iso) (cm)
headScan = 1; %Used to alter computation for head scans. For head scans enter 1 or "true", else for body scans enter 0 or "false".
startingAngle = 0; %Starting gantry angle in degrees
finalAngle = 360; %Final gantry angle in degrees
numAnglesSimmed = 360; %Number of projections/angles to be used in the simulation
numAnglesTrue = 360; %Number of projections/angles in the actual scan
oblique = 0; %Used by PCXMC to simulate oblique angles.
subFieldCoords = [0, 0, 0, 0,0, 0, 0 ; -15 -10, -5, 0, 5, 10, 15 ]; %Sub-field coordinates defined a gantry 0, eg. [0, 0, 0, ... ; y1, y2, y3, ...]
subFieldWidths = [5,5,5,5,5,5,5]; %Width of each sub-field at each sub-field coordinate (cm)
width = 20.6; %Width of the field at isocentre in the sup-inf direction (cm)
kerma = (numAnglesTrue/numAnglesSimmed).*[0.1,0.2,0.5,1,0.5,0.2,0.1]; %Air kerma per projection at each sub-field coordinate (mGy).
filtration = [0.1,0.2,0.5,1,0.5,0.2,0.1]; %Total filtration at each sub-field coordinate (mm Al)

%Phantom geometry details
pWidth = 15; %Dual variable: If headScan = false , this represents the phantom's body width (cm). If headScan = true, anterior radius of phantom's head (cm)
pDepth = 15; %Phantom's body depth (cm)
pWidthTemp = pWidth; % Value as above for destructive use.
pDepthTemp = pDepth; % Value as above for destructive use.
pHeadRadii2 = 13; %If headScan = true, posterior radius of phantoms head (cm).
height = 172; %Phantom's height (cm)
mass = 95; %Phantom's mass (kg)
age = 30; %Phantom's age (years)

%Other Options
plotting = true; %Setting false will disable graphs
interpolation = false; % Setting true will enable linear interpolation of kerma and filtration for points that have been shifted. I.e. value is updated based on new sub-field location, rather than just shifted
resolutionScaling = false; %Can be used to scale the resolution of the entered sub-field data up or down to a user defined number. Uses linear interpolation.
numSubFields = 11; %The number of sub-field coordinates to scale to. Value is only used if resolution scaling is true. Odd numbers are recommended to ensure a point is calculated on CAX
%%%%%%%%%%%%% INPUT DATA END  %%%%%%%%%%%%%%
```

MATLAB Output

- You're mainly going to just need to change the input data to get the outputs you need for PCXMC.
- Input variables are all in the input data section and are described. This section was shown on the page prior.
- Main variable you need for export is "PCXMCInput".
 - This is a matrix that is consistent with input requirements for the PCXMC excel sheet.
 - You can double click to open the variable, click the top left corner of the cell numbers, then copy and paste into "AutocalcRotation-sheet.xls".
- Once data is in PCXMC the instructions for simulating the fields are included with PCXMC documentation. Once simulated you just need to summate the results
- If you run into any troubles getting it to work, happy to help